Proceedings of High Performance Computing Course Project
ID5130
May 5, 2026, Chennai, India

ID5130-PARALLEL-OS-UTILS

A DISTRIBUTED AND PARALLEL IMPLEMENTATION OF CORE OS UTILITIES USING OPENMP AND OPENMPI

Paurush Kumar
Roll No. NA22B002
Indian Institute of Technology Madras
Chennai, Tamil Nadu, India

Akshay Pratap Singh
Roll No. CE21B006
Indian Institute of Technology Madras
Chennai, Tamil Nadu, India

ABSTRACT

This work presents `pfind` and `pgrep`, parallel C++ replacements for common file-system search utilities. The tools are implemented in three modes: a project serial baseline, an OpenMP-only version, and a hybrid OpenMPI+OpenMP version. POSIX directory traversal is used for low-overhead path discovery, OpenMP parallelizes seed-subtree traversal and file scanning, and MPI distributes seed work across ranks. For content search, `pgrep` uses buffered fixed-string matching for small files and streaming Boyer-Moore search for files larger than 1 MiB. Two synthetic workloads were evaluated: a deep many-small-file tree and a large-content tree. Results show clear speedup against the project serial baseline, OpenMP wall-clock speedups over GNU `find`/`grep` in several configurations, and hybrid MPI+OpenMP internal-time scaling, with wall-clock MPI performance limited by launch overhead.

NOMENCLATURE

T_1 Runtime of the project serial implementation
T_p Runtime with p parallel workers or a rank-thread configuration
S_p Speedup, $S_p = T_1/T_p$
E_p Parallel efficiency, $E_p = S_p/p$
B Bytes read by `pgrep`
R MPI rank count
N_t OpenMP thread count per rank
MPI Message Passing Interface
OpenMP Shared-memory directive and runtime API
POSIX Portable Operating System Interface

INTRODUCTION

The Unix utilities `find` and `grep` are standard tools for locating files and searching file contents. They are highly optimized production utilities, but their normal command-line use is still fundamentally a single-process execution model. Modern systems, however, commonly provide many CPU cores, and cluster environments provide multiple nodes. The goal of this project was to explore whether high-performance computing techniques can be applied to this operating-system style workload.

This report describes two tools. The first, `pfind`, searches file and directory names in a recursive directory tree. In the experiments it is compared with GNU `find -name`. The second, `pgrep`, searches file contents recursively and reports the names of matching files. It is compared with GNU `grep -RIl`. The final implementation differs from the original project abstract in two important ways. First, the final evaluation was performed locally with MPI ranks on the same machine rather than on a two-node Ethernet cluster. Second, `pgrep` no longer loads all large files into memory; large files are searched by streaming chunks with overlap.

The main contributions are: (i) separate serial, OpenMP, and MPI+OpenMP binaries for fair comparison; (ii) POSIX-based traversal using `opendir`, `readdir`, and `lstat`; (iii) OpenMP parallel traversal and file-level content search; (iv) MPI seed-work distribution; (v) streaming Boyer-Moore search for large-file fixed-string matching; and (vi) repeatable benchmarking with wall-clock time, internal tool time, match counts, and throughput.

BACKGROUND

GNU `find` searches a directory hierarchy and evaluates expressions such as name tests on each discovered path [**?**]. GNU `grep` searches input for lines or files containing a pattern; in this work the reference command is `grep -RIl`, which recursively searches files and prints only file names containing a match [**?**].

OpenMP provides a shared-memory programming model for C/C++ and Fortran, allowing loop iterations or tasks to be distributed across threads on one node [**?**]. MPI provides a distributed-memory programming model in which independent processes communicate explicitly [**?**]. These models are complementary: MPI can distribute independent subtrees across ranks, while OpenMP can exploit cores within each rank.

For fixed-string content search, large files use the C++17 `std::boyer_moore_searcher`, which implements the Boyer-Moore string searching algorithm [**?**]. Boyer and Moore's original algorithm preprocesses the pattern and often skips ahead by multiple characters during mismatches, reducing the average number of examined text characters for practical search workloads [**?**]. In this project the algorithm is not reimplemented manually; it is used through the C++ standard library searcher interface.

PROBLEM STATEMENT AND OBJECTIVES

The problem is to accelerate recursive file-system queries while preserving the basic output semantics used for benchmarking. For `pfind`, the query is a substring match on the base name of each file or directory. For `pgrep`, the query is a fixed text pattern in file contents, with optional regex support retained as a slower path.

The objectives are:

1. Build working serial, OpenMP-only, and MPI+OpenMP versions of both tools.
2. Compare parallel performance against the project serial baseline.
3. Use GNU `find` and GNU `grep` as external reference baselines.
4. Report both end-to-end wall-clock time and internal traversal/search time.
5. Evaluate two different workload types: deep traversal and large-content search.
6. Validate correctness by comparing match counts and outputs against GNU utilities.

IMPLEMENTATION

Program Variants

The build produces six executables. `pfind_serial` and `pgrep_serial` are compiled without OpenMP and MPI and serve as the primary speedup baselines. `pfind_omp` and `pgrep_omp` use OpenMP but are launched directly, without `mpirun`. `pfind` and `pgrep` are the hybrid MPI+OpenMP binaries and are launched with `mpirun -np R`. Therefore, in the plots, labels of the form $1 \times N_t$ for OpenMP binaries denote one process and N_t threads, not an MPI launch.

Traversal

Both tools first decompose the input roots into seed work. The traversal hot path uses POSIX interfaces. Metadata is obtained by `lstat`; directories are opened with `opendir`; children are enumerated by `readdir` [**?**]. This avoids the overhead and reduced control of `std::filesystem::directory_iterator` in the inner traversal loop. `std::filesystem::path` is still used as a path container, but not for directory iteration.

The local traversal supports breadth-first or depth-first behavior using a deque of pending paths. OpenMP parallelizes traversal over seed work items using dynamic scheduling. Each thread accumulates local matches and metrics, then merges them into rank-local totals.

MPI Work Distribution

Rank 0 expands the top of the tree into seed work items. These seed paths are distributed round-robin across MPI ranks. Each rank then traverses only its assigned seed paths using OpenMP. After local work completes, matching path strings and metrics are gathered back to rank 0. Metrics are reduced using sums for counts and bytes, and the maximum rank time for internal elapsed time.

Content Search

`pgrep` separates file discovery from content search. First, traversal returns candidate regular files. Second, those files are searched in parallel with OpenMP. For files up to 1 MiB, the implementation reads the file into a string and uses the standard fixed-string search path. For files larger than 1 MiB, the implementation reads 1 MiB chunks, keeps an overlap of $m-1$ bytes where m is the pattern length, and applies Boyer-Moore search to each chunk window. The overlap is necessary because a match may begin near the end of one chunk and finish in the next chunk.

Regex mode is supported by `--regex`, but benchmark runs use fixed-string mode. Regex is deliberately isolated behind a branch so it does not affect the fixed-string benchmark path.

ALGORITHMS

Figure **??** summarizes the hybrid algorithm.

The project reports speedup and efficiency as follows.

Hybrid pfind/pgrep workflow
1. Rank 0 receives input roots.
2. Rank 0 expands roots into seed work items.
3. Seed items are distributed round-robin to ranks.
4. Each rank starts N_t OpenMP threads.
5. Threads traverse assigned seed paths using BFS/DFS.
6. For `pfind`, filename predicates are applied during traversal.
7. For `pgrep`, files are collected and then searched in parallel.
8. Matching paths are gathered to rank 0.
9. Metrics are reduced and one CSV row is written.

FIGURE 1. PSEUDOCODE SUMMARY OF THE HYBRID MPI+OPENMP SEARCH WORKFLOW.

Speedup is

$$S_p = \frac{T_1}{T_p}, \tag{1}$$

Parallel efficiency is

$$E_p = \frac{S_p}{p}. \tag{2}$$

For OpenMP-only runs, $p = N_t$. For hybrid runs, a natural worker count is $p = RN_t$, although the plots report configurations as $R \times N_t$ because MPI startup overhead and rank-level work distribution make pure worker-count efficiency less direct. For context, Amdahl's fixed-size model is

$$S_A(p) = \frac{1}{f + (1-f)/p}, \tag{3}$$

where f is the serial fraction. Gustafson-Barsis scaled speedup is

$$S_G(p) = p - \alpha(p-1), \tag{4}$$

where α is the serial fraction in the scaled workload. Since the experiments in this report keep each dataset fixed and vary ranks/threads, the results are primarily strong-scaling measurements and are most directly interpreted with Eqn. (**??**) and Eqn. (**??**). The Karp-Flatt serial fraction,

$$e = \frac{1/S_p - 1/p}{1 - 1/p}, \tag{5}$$

is useful for diagnosing overhead, synchronization, and nonideal scaling when S_p is below p.

TABLE 1. CALCULATED PARALLEL METRICS FOR REPRESENTATIVE RUNS.

Run	p	T_p	S_p	E_p
Deep `pfind_omp` 1×8	8	0.0546	4.74	0.59
Deep `pfind` 2×4	8	0.0805	3.21	0.40
Deep `pgrep_omp` 1×8	8	0.2434	3.48	0.44
Deep `pgrep` 2×4	8	0.3058	2.77	0.35
Large `pgrep_omp` 1×8	8	0.2298	8.13	1.02
Large `pgrep` 2×4	8	0.4470	4.18	0.52

TABLE 2. CALCULATED `pgrep` INTERNAL THROUGHPUT.

Dataset and run	T (s)	Q (GB/s)
Deep serial	0.8479	0.84
Deep OpenMP 1×8	0.2434	2.94
Deep MPI+OpenMP 2×4	0.3058	2.34
Large serial	1.8682	2.00
Large OpenMP 1×8	0.2298	16.25
Large MPI+OpenMP 2×4	0.4470	8.35

The `pgrep` throughput is

$$Q = \frac{B}{T}, \tag{6}$$

where B is bytes read and T is either wall-clock or internal elapsed time.

Table **??** gives the calculated strong-scaling metrics for representative final runs. The speedups use internal elapsed time so that the measured value reflects traversal and search work rather than MPI process-launch time. The throughput values use Eqn. (**??**) and are shown only for `pgrep`, since `pfind` does not read file contents.

EXPERIMENTAL METHODOLOGY

Datasets

Two synthetic datasets were used. The first, `deep_synthetic`, is traversal-heavy: depth 7, fanout 4, four files per directory, and approximately 8 KiB per file. It contains 21,845 directories and 87,380 files. This dataset stresses recursive traversal and many-small-file overhead.

The second, `large_content`, is content-search-heavy: depth 5, fanout 3, four files per directory, and file sizes randomly selected from 1,025 KiB to 4,096 KiB. It contains 364 directories and 1,456 files, totaling approximately 3.6 GiB. Since every file is larger than 1 MiB, this dataset exercises the streaming Boyer-Moore path in `pgrep`.

Benchmark Modes

Each dataset was benchmarked with GNU reference utilities, project serial binaries, OpenMP binaries, and

TABLE 3. SELECTED RESULTS ON THE DEEP SYNTHETIC DATASET.

Config.	Wall	Int.	S_p
`pfind_serial`	0.2626	0.2586	1.00
`pfind_omp` 1×8	0.0602	0.0546	4.74
`pfind` 2×4	0.3845	0.0805	3.21
`pgrep_serial`	0.8586	0.8479	1.00
`pgrep_omp` 1×8	0.2589	0.2434	3.48
`pgrep` 2×4	0.6147	0.3058	2.77

MPI+OpenMP binaries. Thread counts were $N_t = \{1,2,4,8\}$ and MPI ranks were $R = \{1,2\}$ for the hybrid binary. Each configuration was repeated three times. A warmup run was performed before timing to avoid cold-cache distortion in GNU reference baselines.

The benchmark CSV records wall-clock elapsed time around the whole command, internal elapsed time reported by the tool, directories visited, files visited, bytes read, matches, and errors. Correctness was checked by comparing all variants against GNU `find` and `grep`; all final runs had matching counts and zero errors.

RESULTS

Deep Traversal Dataset

The deep traversal workload contains many directories and small files. Table **??** reports selected results. GNU `find` averaged 0.1349 s, while GNU `grep` averaged 0.4969 s after warmup. The OpenMP-only implementations beat GNU wall-clock time at higher thread counts: `pfind_omp` at 8 threads reached 0.0602 s, and `pgrep_omp` at 8 threads reached 0.2589 s.

Figure **??** shows wall-clock runtime by configuration. The pure OpenMP curves fall below the GNU baselines for the best `pfind` and `pgrep` configurations. The hybrid MPI+OpenMP wall-clock curves improve with more work but remain affected by `mpirun` launch overhead.

Internal timing removes process-launch effects. In that view, the hybrid implementation shows clearer scaling. For example, `pgrep` at 2×4 reaches 0.3058 s internal time, faster than GNU `grep`'s 0.4969 s reference time. The internal-time plot is generated in the final plot directory, while Table **??** reports the calculated speedup values used for analysis.

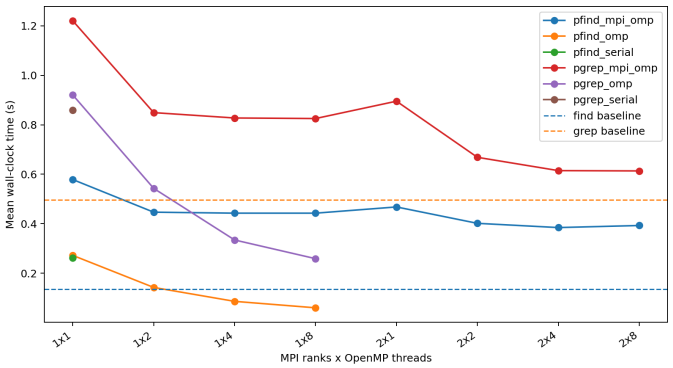

FIGURE 2. WALL-CLOCK RUNTIME ON THE DEEP SYNTHETIC DATASET.

Large Content Dataset

The large-content dataset stresses content scanning rather than directory traversal. GNU `grep` averaged 1.0936 s. The OpenMP `pgrep` implementation achieved substantial gains: 0.3948 s at 4 threads and 0.2323 s at 8 threads. The best internal speedup against `pgrep_serial` was 8.13 at 8 OpenMP threads. Hybrid `pgrep` also beat GNU wall-clock time for several configurations, including 2×4 with 0.7466 s.

Figure **??** shows wall-clock runtime on the large-content workload. Unlike the deep dataset, `pgrep` has enough per-file work for the parallel search implementation to overcome overhead more clearly.

Figure **??** reports `pgrep` throughput. The best internal throughput was approximately 16.2 GB/s for `pgrep_omp` at 8 threads, compared with about 2.0 GB/s for the project serial implementation. This result demonstrates the benefit of file-level parallel scanning and the large-file fixed-string path.

TABLE 4. SELECTED RESULTS ON THE LARGE-CONTENT DATASET.

Config.	Wall	Int.	S_p
`pgrep_serial`	1.8697	1.8682	1.00
`pgrep_omp` 1×2	0.7708	0.7689	2.43
`pgrep_omp` 1×4	0.3948	0.3929	4.76
`pgrep_omp` 1×8	0.2323	0.2298	8.13
`pgrep` 2×4	0.7466	0.4470	4.18

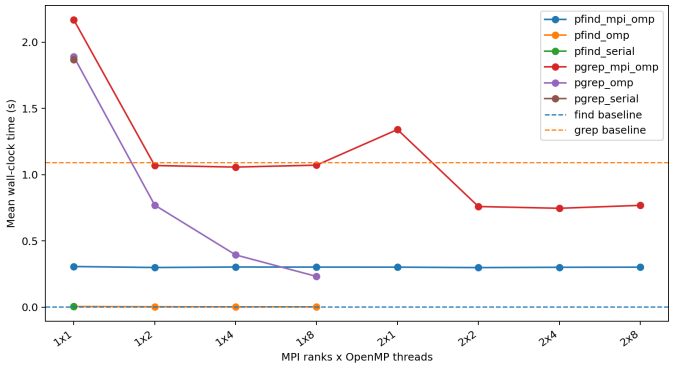

FIGURE 3. WALL-CLOCK RUNTIME ON THE LARGE-CONTENT DATASET.

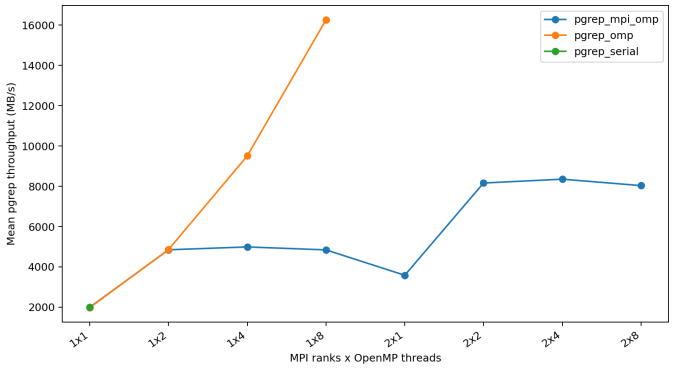

FIGURE 4. INTERNAL `pgrep` THROUGHPUT ON THE LARGE-CONTENT DATASET.

DISCUSSION

The results show that the best parallel strategy depends on workload structure. The deep dataset has many small files and many directories. In this case, traversal and file-open overhead dominate, and OpenMP gives a strong benefit by processing independent seed paths and files concurrently. MPI internal time also improves, but end-to-end wall-clock time remains limited by launch overhead for single-query runs. For example, `pfind_omp` at 8 threads gives $S_p = 4.74$ and $E_p = 0.59$, while `pgrep_omp` at 8 threads gives $S_p = 3.48$ and $E_p = 0.44$ on the deep dataset.

The large-content dataset has fewer directories but much larger files. Here, `pgrep` benefits more from content-scanning parallelism and streaming fixed-string search. The OpenMP-only version performs especially well because it avoids MPI startup while still using all local cores. Hybrid MPI+OpenMP becomes more attractive when measuring internal work time or when the same MPI allocation would process multiple queries without repeated launch overhead. On this workload, `pgrep_omp` at 8 threads gives $S_p = 8.13$ and $E_p = 1.02$ relative to project serial internal time. This slightly superlinear result should be interpreted as a cache and I/O scheduling effect rather than ideal algorithmic scaling. The hybrid `pgrep` 2×4 run gives $S_p = 4.18$ with $p = 8$, so $E_p = 0.52$; the corresponding Karp-Flatt fraction is about 0.13, indicating measurable overhead from MPI launch, work distribution, and shared file-system contention.

The project serial baseline is the primary speedup baseline because it isolates the effect of added parallelism. GNU `find` and `grep` remain important reference baselines because they represent optimized production tools. The final results are therefore best interpreted in two layers: speedup over project serial demonstrates parallel scalability, while comparison against GNU utilities shows where the prototype is already competitive.

LIMITATIONS

The MPI scheduler is static: seed work is distributed round-robin after rank 0 expands the top of the tree. Highly imbalanced directory trees could leave some ranks idle while others continue processing large subtrees. Dynamic work stealing would address this limitation. In addition, MPI launch overhead is included in wall-clock timing; this is realistic for single-shot command-line use but unfavorable to MPI. A persistent worker mode could amortize this overhead across multiple queries.

The large-content benchmark uses synthetic random text rather than a real document corpus. This is useful for controlled size and match placement but may not fully represent real-world compression, caching, or text distributions. Finally, regex support is functional but was not the focus of performance experiments.

CONCLUSIONS

This project implemented and evaluated parallel versions of `find` and `grep`-style workloads using C++, OpenMP, and OpenMPI. The final tools provide serial, OpenMP-only, and MPI+OpenMP modes, allowing fair separation of serial baseline, shared-memory scaling, and hybrid distributed-memory scaling.

On the deep traversal dataset, OpenMP `pfind` and `pgrep` beat GNU wall-clock baselines at higher thread counts, while

MPI+OpenMP showed strong internal-time scaling. On the large-content dataset, `pgrep` achieved substantial speedup, with the 8-thread OpenMP run reaching 0.2323 s wall-clock time versus 1.0936 s for GNU `grep`. The main conclusion is that parallel file-system search can be competitive for controlled workloads, especially when many independent files or large file contents expose enough work to amortize threading overhead. MPI is useful for distributing work and improving internal compute time, but launch overhead must be addressed for interactive single-query use.

ACKNOWLEDGMENT

The authors thank the course staff for the project structure and evaluation rubric. The implementation used open-source MPI, OpenMP, GNU, and C++ standard library tooling.

REFERENCES

[1] Free Software Foundation, 2026, "GNU Findutils Manual," GNU Project, available at `https://www.gnu.org/software/findutils/manual/`.

[2] Free Software Foundation, 2025, "GNU Grep Manual," GNU Project, available at `https://www.gnu.org/software/grep/manual/grep.html`.

[3] OpenMP Architecture Review Board, 2024, "OpenMP API Specification," available at `https://www.openmp.org/specifications/`.

[4] Message Passing Interface Forum, 2025, "MPI Standard Documents," available at `https://www.mpi-forum.org/docs/`.

[5] The Open Group, "`opendir` and `readdir` POSIX Interfaces," The Open Group Base Specifications, available at `https://pubs.opengroup.org/onlinepubs/`.

[6] cppreference.com, 2026, "`std::boyer_moore_searcher`," available at `https://en.cppreference.com/w/cpp/utility/functional/boyer_moore_searcher.html`.

[7] Boyer, R. S., and Moore, J. S., 1977, "A Fast String Searching Algorithm," *Communications of the ACM*, 20(10), pp. 762–772.